

Computer Vision Homework 6

NTU CSIE D13922014 黃丰楷

Yokoi Connectivity Number

- Description

This program calculates the Yokoi connectivity number for a downsampled version of lena.bmp. It begins by loading the image in grayscale and binarizing it: pixels with values greater than 127 are set to 255, while others are set to 0. The image is then downsampled to 64x64 by taking the top-left pixel from each 8x8 block.

The function `num` is defined to classify connectivity relations, returning "q," "r," or "s" based on the relationships of neighboring pixels. The main function `YokoiConnectivityNumber` uses these classifications for each pixel and its neighbors, computing the connectivity number according to 4-connectivity. Specifically, a pixel is assigned a value of 5 if all neighboring regions are classified as "r"; otherwise, it receives the count of "q" classifications among its neighbors.

The result is stored in a 64x64 matrix where only foreground pixels are processed. Finally, the connectivity numbers are saved in `result.txt` as a 64x64 matrix.

- Code

```
import numpy as np
from PIL import Image
import os
import cv2

ori_img = cv2.imread("./lena.bmp", cv2.IMREAD_GRAYSCALE)
try:
    os.mkdir("./output")
    cv2.imwrite('./output/ori.png', ori_img)
except:
    cv2.imwrite('./output/ori.png', ori_img)

binary_img = np.zeros(ori_img.shape, dtype=int)
binary_img[binary_img > 127] = 255

downsampled_img = np.zeros((64, 64), dtype=int)
for i in range(64):
    for j in range(64):
        downsampled_img[i, j] = binary_img[8 * i, 8 * j]

def num(b, c, d, e):
    if b == c and (d != b or e != b):
```

```

        return "q"
    if b == c and (d == b and e == b):
        return "r"
    return "s"

def YokoiConnectivityNumber(img, i, j):
    x0 = img[i, j]
    x1 = img[i, j + 1] if j + 1 < img.shape[1] else 0
    x2 = img[i - 1, j] if i - 1 >= 0 else 0
    x3 = img[i, j - 1] if j - 1 >= 0 else 0
    x4 = img[i + 1, j] if i + 1 < img.shape[0] else 0
    x5 = img[i + 1, j + 1] if i + 1 < img.shape[0] and j + 1 <
img.shape[1] else 0
    x6 = img[i - 1, j + 1] if i - 1 >= 0 and j + 1 < img.shape[1] else
0
    x7 = img[i - 1, j - 1] if i - 1 >= 0 and j - 1 >= 0 else 0
    x8 = img[i + 1, j - 1] if i + 1 < img.shape[0] and j - 1 >= 0 else
0

    a1 = num(x0, x1, x6, x2)
    a2 = num(x0, x2, x7, x3)
    a3 = num(x0, x3, x8, x4)
    a4 = num(x0, x4, x5, x1)

    if a1 == 'r' and a2 == 'r' and a3 == 'r' and a4 == 'r':
        return 5
    else:
        return sum([a == 'q' for a in [a1, a2, a3, a4]])

output_matrix = np.full(downsampled_img.shape, ' ', dtype=str)

for i in range(downsampled_img.shape[0]):
    for j in range(downsampled_img.shape[1]):
        if downsampled_img[i, j] == 255:
            output_matrix[i, j] =
str(YokoiConnectivityNumber(downsampled_img, i, j))

output_dir = "output"
os.makedirs(output_dir, exist_ok=True)
with open(os.path.join(output_dir, "result.txt"), "w") as fp:
    for row in output_matrix:
        fp.write(" ".join(row) + "\n")

```

- Result

3 / 3